



**NATIONAL JUNIOR COLLEGE
SENIOR HIGH 2 PRELIMINARY EXAMINATION
HIGHER 2**

COMPUTING

9569/02

Paper 2 (Lab-based)

31 August 2026

3 hours

Additional Materials: Electronic version of BICYCLE.TXT data file
 Electronic version of DECK.CSV data file
 Electronic version of HAND.CSV data file
 Electronic version of PROCESS.TXT data file
 Electronic version of REPAIR.TXT data file
 Electronic version of STATION.TXT data file
 Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6 marks out of 100** will be awarded for the use of common coding standards for programming style.

The number of marks is given in the brackets [] at the end of each question or part question. The total number of marks for this paper is 100.

This document consists of **17** printed pages.

Instruction to candidates:

Your program code and output for each of Questions 1 to 4 should be saved in a single .ipynb file using Jupyter Notebook. For example, your program code, output and answer for Question 1 should be saved as:

TASK1_<your name>.ipynb

Make sure that each of your .ipynb files shows the required output in Jupyter Notebook.

1 Name your Jupyter Notebook as:

TASK1_<your name>.ipynb

A student robotics team sends short messages from a controller to a robot during a competition. Each message must be encrypted before transmission and encoded as binary data. A checkdigit is added to each encoded character so that some transmission errors can be detected.

Only the following 37 characters may appear in a message:

- The space character " " has code 0
- the rest of the 36 characters are ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789. A has code 1, B has code 2, and so on. The digit 9 has code 36.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        Program Code
```

Output:

Task 1.1

Write program code for a function `to_six_bit(ch)` that:

- takes a valid character `ch` as a parameter
- returns the 6-bit binary string for an integer code from 0 to 36

The use of inbuilt function to convert an integer to binary is not allowed for this question.

For example,

- `to_six_bit('A')` returns 000001.
- `to_six_bit('0')` returns 011011.

[5]

Task 1.2

Even parity is used. A parity bit is appended to the end of each 6-bit binary string so that the final 7-bit string contains an even number of 1 bits.

Write program code for a function `add_even_parity(six_bit)` that:

- takes a 6-bit binary string `six_bit` as a parameter
- returns the 7-bit string with the parity bit added

[3]

Task 1.3

Before binary encoding, each character code is encrypted using the following cipher:

$$E(x) = (5x + 8) \bmod 37$$

x is the original character code and $E(x)$ is the encrypted character code.

For example, if F is the original character, then the encrypted character code is

$$5 \times 6 + 8 = 38 \equiv 1 \bmod 37.$$

Write a function `char_encrypt_to_seven_bit(ch)` that:

- takes in a character `ch` as parameter
- converts the character to uppercase if applicable
- ignores any character that is not in the valid character set
- converts each valid character to its encrypted character code
- converts each encrypted code to a 6-bit binary string
- adds an even parity bit to produce a 7-bit binary string
- returns the 7-bit binary string

[5]

Use your function to encode the message ?TURN LEFT TO FIND THE CHEESE!, character by character, and write the resulting 7-bit strings to a text file named `SIGNAL.TXT`, one 7-bit string per line.

[2]

Task 1.4

Let $D(x)$ be the inverse cipher of $E(x)$. In other words, if x is the original character code, $E(x)$ is the encrypted character code then

- $D(E(x)) = x$, and
- $E(D(x)) = x$

Write program code `inverse_cipher(ch)` that implements the $D(x)$. In other words, it:

- takes in a valid encrypted character code `ch`, and
- return the decrypted character code

[1]

2 Name your Jupyter Notebook as:

TASK2_<your name>.ipynb.

An operating system stores records about data held in virtual memory. Each record contains:

- a positive integer process ID, `process_id`, which identifies a process
- a positive integer virtual address, `virtual_address`
- a string, `data`, containing the data stored at that virtual address.

The combination of `process_id` and `virtual_address` uniquely identifies a record. For this question, you may assume that no duplicate combination of `process_id` and `virtual_address` will be inserted.

The records are stored in a hash table. The hash table is implemented as a one-dimensional list of buckets. Each bucket contains a linked list. When two records have the same hash value, the collision is resolved by storing both records in the linked list at that bucket. This method of resolving collisions is called **separate chaining**.

For this question, you do not need to create getter and setter methods for the class's attributes unless it is specified by the question.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

In [1]: *#Task 2.1*
Program Code

Output:

Task 2.1

A node in a linked list stores one virtual-memory record.

Write program code to declare the class `Node` and its constructor. The constructor must:

- take `process_id`, `virtual_address` and `data` as parameters
- store the three parameter values in attributes
- initialise an attribute `next` to `None`.

[3]

Task 2.2

Write program code to declare the class `LinkedList`.

The class must have:

- a constructor that initialises an attribute `head` to `None`
- a method `insert(process_id, virtual_address, data)` that:
 - takes `process_id`, `virtual_address` and `data` as parameters
 - creates a new `Node`
 - stores the new node at the end of the linked list
 - returns the Boolean value `True` when the node has been inserted.

[5]

Task 2.3

Add a method `find(process_id, virtual_address)` to the class `LinkedList`.

The method must:

- take `process_id` and `virtual_address` as parameters
- traverse the linked list from its head
- return the string data item from the node with the matching process ID and virtual address
- return the Boolean value `False` if the record is not stored in the linked list.

[3]

Task 2.4

Write program code to declare the class `HashTable`.

The class must have:

- a constructor that:
 - takes the size of the hash table as a parameter
 - stores the size in an attribute
 - initialises the hash table as a one-dimensional list containing a new empty `LinkedList` object at every index
- a method `calculate_hash(process_id, virtual_address)` that:
 - takes `process_id` and `virtual_address` as parameters
 - calculates and returns the hash value using the following hashing algorithm. The hashing algorithm is:

$$\text{hash}(p, v) = (31p + v) \mod s$$

where:

- * p represents the process ID
- * v represents the virtual address
- * s represents the size of the hash table.

[4]

Task 2.5

Add the following methods to the class HashTable:

- `insert(process_id, virtual_address, data)` must:
 - takes `process_id`, `virtual_address` and `data` as parameters
 - calculates the index using the `calculate_hash()` method
 - inserts the record into the linked list at the calculated index
 - returns the value returned by the linked list's `insert()` method
- `find_record(process_id, virtual_address)` must:
 - take `process_id` and `virtual_address` as parameters
 - calculate the index using the `calculate_hash()` method
 - return the string data item for the matching record, or return `False` if the record is not stored in the hash table
- `display()` must output every non-empty bucket index followed by all the records in its linked list. The records must be output in linked-list order. An example of the output is given below.

Bucket 2

```
[2026, 1, 'Test data from pid 2026-1']
```

Bucket 3

```
[2026, 2, 'Test data from pid 2026-2'] -> [16002, 11, 'Test data from pid 16002-11']
```

[6]

Task 2.6

The text file `PROCESS.TXT` stores virtual-memory records. The first line contains field names. Each remaining line contains a process ID, a virtual address and a string data item, separated by commas.

Write program code for the function `read_data(HashTable, filename)` to:

- take a `HashTable` object `ht` and a filename `filename` as parameters
- read each record from the text file, excluding the line containing the field names
- convert the process ID and virtual address to integers
- call the hash table's `insert()` method with each process ID, virtual address and string data item.

Write program code to:

- create a `HashTable` object with size 5
- call `read_data()` using the new hash table and `PROCESS.TXT`
- call `find_record()` with process ID 16002 and virtual address 12, and output the returned string data item in an appropriate message
- call `find_record()` with process ID 9999 and virtual address 1, and output 'Record not found' when the function returns `False`.

[5]

3 Name your Jupyter Notebook as:

TASK3_<your name>.ipynb.

A card game, Galactic Duel, is played using a deck of 120 cards. Each player will draw 6 cards from the deck to form a hand.

Each card represents a character in the fictional Star Wars universe with the following attributes: `species` (Human, Droid, Wookiee), `weapon` (Bare Hands, Blaster, Bowcaster, Lightsaber), and base strength (1 to 5). Every combination of species, weapon, and strength appears at least once in the deck.

A card's true battle power is its **power value**, calculated as:

$$\text{power value} = \text{base strength} \times \text{species multiplier} \times \text{weapon multiplier}$$

The species and weapon multipliers depend on the character's species and its primary weapon. They are fixed as follows:

Species Multiplier

Species	Multiplier
Human	1.3
Droid	1.5
Wookiee	1.7

Weapon Multiplier

Weapon	Multiplier
Bare Hands	1.4
Blaster	1.8
Bowcaster	1.9
Lightsaber	2.0

In this task you are not allowed to use any of Python's built-in sort functions.

Task 3.1

Write a function `quicksort(data)` that:

- takes in a list of tuples `data` as a parameter
- uses a quicksort algorithm to sort a list of tuples in descending order, using the last element of each tuple as the sort key. Tuples with equal sort keys may appear in any order relative to one another.
- uses a randomly chosen pivot at each partitioning step
- returns the sorted list.

[6]

Task 3.2

Write program code to read the card deck from the file `DECK.CSV`. The first row contains field names. Each subsequent row contains a card's name, species, weapon, and base strength, in that order.

For every card, calculate the card's power value using the formula and multiplier tables given above, and store each card as a tuple `(name, species, weapon, base_strength, power_value)`, where:

- `name` is the card's name
- `species` is the character's species
- `weapon` is the primary weapon of the card character
- `base_strength` is the base strength of the card character
- `power_value` is calculated using the formula and multiplier tables given above.

Print the deck, one card per line, showing the card's name, species, weapon, base strength, and power value.

Print each base strength as an integer and each power value rounded to one decimal place.

[4]

Task 3.3

The median of a sorted list of values is defined as:

- the middle value, if the number of values is odd.
- the average of the two middle values, if the number of values is even.

Write a function `closest_to_median(species_name, deck)` that

- takes two parameters:
 - a string `species_name` representing a species name, and
 - a list `deck` representing a deck
- searches the list for a card of the specified species whose power value is closest to the median power value for that species.
- prints the median power value.
- prints the card's name, species, weapon, base strength and power value. All non-integer numerical values should be printed by rounding to 1 decimal place.
- If more than one card is equally close to the median, print any of those cards. [6]

Sample output:

For species Human, median power value is 7.2

One such card is ('Mace', 'Human', 'Blaster', 3, 7.0)

Test your program with the deck that you created in Task 3.2 and all possible species given. [1]

Task 3.4

Read a hand of 6 cards from a CSV file `HAND.csv`. The first row contains field names. Each subsequent row contains a card's name, species, weapon, and base strength, in that order.

From this hand, find a combination of exactly 3 cards such that all three cards have different species and all three cards have different weapons, such that the sum of their power values is the maximum possible among all such valid combinations.

If such a combination exists, print each chosen card's name, species, weapon, base strength, and power value, together with the total power value of the three cards. Print each base strength as an integer. Print each card's power value and the total power value rounded to one decimal place.

If there is more than one such combination, any one of them is acceptable.

If no valid combination exists in the hand, print `'No valid combination'`.

Sample output:

Cards:

- Palpatine Human Lightsaber 5 13.0
- BT-1 Droid Blaster 5 13.5
- Salporin Wookie Bowcaster 5 16.1

Total power value: 42.6

[9]

4 Name your Jupyter Notebook as:

TASK4_<your name>.ipynb.

A university club rents bicycles to students for short journeys between school venues. The executive committee wants a database to store data about bicycle stations, bicycles, and maintenance work.

The database must contain three tables with the fields shown below.

Station:

- StationID - unique code for the station, for example S01
- StationName - name of the station
- Capacity - maximum number of bicycles that can be assigned to the station

Bicycle:

- BicycleID - unique code for the bicycle, for example B1001
- StationID - station currently assigned to the bicycle
- Model - model name of the bicycle
- Status - one of available, rented, or maintenance

Repair:

- RepairID - unique integer for a repair job
- BicycleID - ID of bicycle being repaired
- Mechanic - name of mechanic assigned
- Fault - short description of the fault
- Cost - repair cost in dollars
- Complete - 0 if repair is incomplete, 1 if repair is complete

The database will be created and tested using sample data provided in the following text files: BICYCLE.TXT, REPAIR.TXT and STATION.TXT.

Each text file contains comma-separated records and does not contain a field-name row. The fields occur in the same order as the corresponding table fields listed above.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

In [1]: *#Task 4.1*
Program Code

Output:

Task 4.1

Write program code that uses SQL to create a database named `Rental.db` with the three tables `Station`, `Bicycle`, and `Repair`.

Define the primary and foreign keys for each table. [6]

Task 4.2

Write program code that uses SQL to insert the data from each of the three text files into the appropriate fields in each database table.

Run your program. [5]

Task 4.3

Write a function `show_available_bicycles(station_name)` that:

- has a string parameter `station_name` that represents the name of a station
- uses SQL to search for all bicycles that are currently available at the station whose name is passed as a parameter.
- for each bicycle found, output the bicycle ID, model, and station name. The output must be sorted in descending order of bicycle ID.
- if no bicycle is found, output "No bicycles found at <station_name>", where <station_name> is the supplied string parameter [4]

Test your function with the station name `Library`. [1]

Task 4.4

Write a function `mechanic_summary()` that:

- uses SQL to check the completion status of the bicycle repairs
- outputs the total incomplete repair cost for each mechanic. If the mechanic has completed all the repairs assigned to them, the mechanic's total incomplete repair cost should be \$0.
- the output must show the mechanic name and the total cost, sorted in descending order of total cost. Mechanics with equal total costs may appear in any order.

For example, if a mechanic named Anya has three repair jobs assigned to her:

- repair ID: 1, cost: \$18.50, incomplete,
- repair ID: 3, cost: \$25.00, complete,
- repair ID: 20, cost: \$14.30, incomplete,

the following line should be output for her.

Sample Output:

Mechanic Name: Anya, Total incomplete repair cost: \$32.80

[4]

Task 4.5

Write a Python program and the necessary files to create a web application. The web application offers the following menu options:

Show Available Bicycles Across All Stations
Incomplete Mechanic Jobs and Total Cost

[1]

Each menu option must link to a separate web page to display the information.

The “Show Available Bicycles Across All Stations” page must display the bicycle ID, model, and station name for every available bicycle across all stations.

The “Incomplete Mechanic Jobs and Total Cost” page must display every mechanic's name and total incomplete repair cost, including a total of zero for a mechanic who has completed all assigned repairs.

The outputs should not be hardcoded into the web pages. You may modify and reuse the functions you write in Task 4.3 and Task 4.4.

Save your program code as

Task4_5_<your name>.py

with any additional files/subfolders as needed in a folder named

Task4_5_<your name>

Run the web application and save the three output pages as:

- Task4_5_<your name>_index.html
- Task4_5_<your name>_all.html
- Task4_5_<your name>_jobs.html

[5]